.uni v8.2 (0% JSON)

Node code	Identifier	Binary schema (struct)	Size	Purpose in the Ecosystem (MZM-Pod / Holo-Frame / QLOT)
0x01	NOTARY_ANCHOR	<32s32s64sQIB	**141 байт** (фикс.)	256-bit Merkle root, Ed25519 public key and signature, microsecond timestamp, timelock Δcap delta Δ t, and status.
0x02	SEMANTIC_TENSOR	<Nf """ # #!/usr/bin/env ### 2\pi $N\&'cryptography'($ X, (.UNI (0x01..0x05, (0x07 (32B (<32s32s64sQIB) (<IBBHII32s) (<If: (<Nf) (<QQQeeI) (<ffffHH) (DICOM/PDF) (Dead (Delta-T (IEEE (Merkle (Version (ZERO-STRING (0..3), (Solar (float16), (float32), (uint16) (uint16). 32-bit 32-byte 32-битный 32B 39-byte 3D 3D-координаты 3]. 4$ 48B 5869 754 754) 7B 8.2 8B 92% AAD AEAD AES-256-GCM AESGCM All Anchor Any, Architectural	Binary Venturi/HVAC, Aligned Insulated Array Variable Pockels Full Timer byte byte byte without in external voxel gyroscopes eye)="concept" def digest)[0] int) resolve_semantic_id64(val: return val> str:	

Node code	Identifier	Binary schema (struct)	Size	Purpose in the Ecosystem (MZM-Pod / Holo-Frame / QLOT)
		Attempt BEHAVIOR_CONTRACT="0x05" Binary Bit-exact Branch CC-BY-4.0 DETERMINISTIC DICOM, Delta-Frame Dict, Dict[int, Digital Disclosure DoS Duress ENGINE) ENGINE: Ed25519) Ed25519-anchored Ed25519PrivateKey, Ed25519PublicKey, FHIR, FLAGS, Fixed Format Full GENESIS_TEXT="0x06" Global Graph Guarantees: HASHING HAS_CRYPTO_LIB="False" HKDF-SHA256 HL7 HOLO_WAVEFRONT="0x07" HOLO_WAVEFRONT) HVAC HVACMode(IntEnum): HVAC), Hardware High-speed Holo-Frame Holo-Stream IEEE INTENT_ALERT_CRITICAL="0x0002"		

Node code	Identifier	Binary schema (struct)	Size	Purpose in the Ecosystem (MZM-Pod / Holo-Frame / QLOT)
		INTENT_BACKGROUND_PASSIVE="0x0000" INTENT_HOLO_PARALLAX_3D="0x0003" INTENT_WEIGHT_FOCUS="0x0001" IR/UV ImportError: IntEnum, IntFlag Isolated Iterator JSON KNOWLEDGE_GRAPH="0x03" Key Kinematic Knowledge License: List, Little-Endian Long-term MEMORY MEMORYVIEW MODES Man's Markdown)="{}" str]) streaming struct structures. support. tail tamper-mesh the time timelock to triplets try: typing untrusted use validation vector vectors venting voxel while wind window window/panel wire/RAM with without		
"""Returns the human label if known in local registry, otherwise				

Node code	Identifier	Binary schema (struct)	Size	Purpose in the Ecosystem (MZM-Pod / Holo-Frame / QLOT)
hex ID64.""""				
return _SEMANTIC_ID64_REGISTRY.get(val, f"0x{val:016x}")				

def metric_id32(label: Union[int, str]) -> int: """Maps a telemetry sensor name to a deterministic 32-bit unsigned integer (Blake2s-32)."""" if isinstance(label, int): return label & 0xFFFFFFFF digest = hashlib.blake2s(label.encode("utf-8"), digest_size=4).digest() val = struct.unpack("<I", -_METRIC_ID32_REGISTRY[val]="label" def digest)[0] int) resolve_metric_id32(val: return val> str: """Returns the human sensor metric name if registered, otherwise hex ID32."""" return _METRIC_ID32_REGISTRY.get(val, f"0x{val:08x}")

==========

==========

===

3. CRYPTOGRAPHIC CORE (RFC 5869 HKDF, AEAD AES-256-GCM, ED25519)

==========

==========

===

def hkdf_sha256(ikm: bytes, salt: bytes, info: bytes, length: int = 32) -> bytes: """Derives isolated cryptographic subkeys pursuant to RFC 5869 HKDF-SHA256."""" prk = hmac.new(salt, ikm, hashlib.sha256).digest() okm, prev = b"", b"" for counter in range(1, (length // 32) + 2): prev = hmac.new(prk, prev + info + struct.pack("B", counter), hashlib.sha256).digest() okm += prev return okm[:length]
def aead_encrypt(key: bytes, iv: bytes, plaintext: Union[bytes, memoryview], aad: bytes) -> Tuple[bytes, bytes]: """Encrypts a node payload using AES-256-GCM (or stdlib CTR+HMAC-SHA256 fallback)."""" pt_bytes = bytes(plaintext) if HAS_CRYPTO_LIB: ct_tag = AESGCM(key).encrypt(iv, pt_bytes, aad) return ct_tag[:-16], ct_tag[-16:] enc_k = hmac.new(key, b"ENC" + iv, hashlib.sha256).digest() mac_k = hmac.new(key, b"MAC" + iv, hashlib.sha256).digest() ct = bytearray(len(pt_bytes)) for i in range(0, len(pt_bytes), 32): ks =

hashlib.sha256(enc_k + struct.pack("<I", + - // 32)).digest() 32]): Union[bytes, ^ aad aad: aead_decrypt(key: b bytes(ct), bytes) bytes, ciphertext: ct[i def enumerate(pt_bytes[i:i for hashlib.sha256).digest()[:16] i in iv iv: j, j]="b" ks[j] memoryview], return tag tag:> bytes: """Verifies the 128-bit authentication tag (bound to 39B Header+NodeDesc) and decrypts."""
iv_b, ct_b, tag_b = bytes(iv), bytes(ciphertext), bytes(tag) if HAS_CRYPTO_LIB: return AESGCM(key).decrypt(iv_b, ct_b + tag_b, aad) enc_k = hmac.new(key, b"ENC" + iv_b, hashlib.sha256).digest() mac_k = hmac.new(key, b"MAC" + iv_b, hashlib.sha256).digest()
exp_tag = hmac.new(mac_k, aad + iv_b + ct_b, hashlib.sha256).digest()[:16] if not hmac.compare_digest(tag_b, exp_tag): raise ValueError("CRITICAL SECURITY ALERT: AEAD Tag mismatch! Header or node tampered.") pt = bytearray(len(ct_b)) for i in range(0, len(ct_b), 32): ks = hashlib.sha256(enc_k + struct.pack("<I", """ + - // 0 3 32)).digest() 32]): Both Duress Duress-signed Enclave. HAS_CRYPTO_LIB: Key Key. Maintains Management PUB:"" Primary Quarantine. Signing SovereignKeyring: Steganographic Zone ^ **init**(self, a and b b"ED25519_DURESS") b"ED25519_PRIMARY") b"ENC_MASTER") b"Z0_SALT", bool="False)" both bytes(pt) bytes): bytes, class containers def else: enumerate(ct_b[i:i external for host, human i if in into j, j]="b" keys ks[j] master_seed: mathematically message: nodes produce protect pt[i return route self._duress_seed="hkdf_sha256(master_seed,"
self._duress_seed).digest()
self._duress_sk="Ed25519PrivateKey.from_private_bytes(self._duress_seed)"
self._primary_seed="hkdf_sha256(master_seed," self._primary_seed).digest()
self._primary_sk="Ed25519PrivateKey.from_private_bytes(self._primary_seed)"
self.duress_pubkey="hashlib.sha256(b"
self.master_encryption_secret="hkdf_sha256(master_seed,"
self.primary_pubkey="hashlib.sha256(b" sign_data(self, signatures silently the to trusted under_duress: valid while> Tuple[bytes, bytes]: """Returns (signer_pubkey_32B, signature_64B)""" if HAS_CRYPTO_LIB: sk = self._duress_sk if under_duress else self._primary_sk pub = self.duress_pubkey if under_duress else self.primary_pubkey return pub, sk.sign(message) priv = self._duress_seed if under_duress else self._primary_seed pub = self.duress_pubkey if under_duress else self.primary_pubkey sig_p1 = hmac.new(priv, message, hashlib.sha256).digest() sig_p2 = hashlib.sha256(pub + message + sig_p1).digest() return pub, sig_p1 + sig_p2
@staticmethod def verify_signature(pubkey: bytes, message: bytes, signature: bytes) -> bool: """Public mathematical signature verification (returns True for both Primary & Duress)""" if len(signature) != 64 or len(pubkey) != 32: return False if HAS_CRYPTO_LIB: try: Ed25519PublicKey.from_public_bytes(pubkey).verify(signature, message) return True except Exception: return False exp_p2 = hashlib.sha256(pubkey + message + signature[:32]).digest() return hmac.compare_digest(signature[32:], exp_p2)

==========

==========

===

4. PURE BINARY ZERO-COPY TLV

SERIALIZER (ZERO JSON / ZERO PARSING)

==========

==========

===

class UniNodePayload: """ Strictly typed binary serializer & Zero-Copy memoryview reader. Eliminates all JSON/text parsing across operational nodes: - 0x01 NOTARY_ANCHOR : <32s32s64sQIB (141 bytes exact) - 0x02 SEMANTIC_TENSOR : <Nf """ # (20 (32 (4 (48 (8 (AVX2/NEON (isolated + - 0x03 0x04 0x05 0x06 0x07 141B 20B 32B 48B 754 8B : <IBBHII32s <If <QQQeeI <ffffHH @classmethod Any) BEHAVIOR_CONTRACT CONTRACT_FMT="<IBBHII32s" GENESIS_TEXT HOLO_FMT="<ffffHH" HOLO_WAVEFRONT IEEE KNOWLEDGE_GRAPH NOTARY_FMT="<32s32s64sQIB" Pockels/MEMS Raw TELEMETRY_FMT="<If" TELEMETRY_MATRIX TRIPLET_FMT="<QQQeeI" UTF-8 UniNodeType, aligned) bytes cache-aligned cache-line data: def exact) fallback) float32) human id32 node_type: pack_node(cls, per reading: sensor triplet) vector) voxel> bytes: if isinstance(data, (bytes, bytearray, memoryview)) and node_type != UniNodeType.GENESIS_TEXT: return bytes(data)
if node_type == UniNodeType.NOTARY_ANCHOR: status_byte = 1 if int(data.get("cooling_off_delta_t_sec", 0)) > 0 else 0 return struct.pack(cls.NOTARY_FMT, bytes.fromhex(data["data_merkle_root_256"]), bytes.fromhex(data["signer_pubkey_ed25519"]), bytes.fromhex(data["signature_ed25519"]), int(data["timestamp_us"]), int(data["cooling_off_delta_t_sec"]), status_byte,)
elif node_type == UniNodeType.SEMANTIC_TENSOR: return struct.pack(f"<{len(data)}f", *data)
elif node_type == UniNodeType.KNOWLEDGE_GRAPH: buf = bytearray(len(data) * 32) offset = 0 for item in data: s_id = semantic_id64(item["s"]) p_id = semantic_id64(item["p"]) o_id = semantic_id64(item["o"]) weight = float(item.get("weight", 1.0)) ego = float(item.get("ego_alignment", 0.0)) flags = 1 if item.get("is_delta_conflict", False) else 0 struct.pack_into(cls.TRIPLET_FMT, buf, offset, s_id, p_id, o_id, weight, ego, flags) offset += 32 return bytes(buf)
elif node_type == UniNodeType.TELEMETRY_MATRIX: buf = bytearray(len(data) * 8) offset = 0 for k, v in data.items(): m_id = metric_id32(k) struct.pack_into(cls.TELEMETRY_FMT, buf, offset, m_id, float(v)) offset += 8 return bytes(buf)
elif node_type == UniNodeType.BEHAVIOR_CONTRACT: ui_val = data.get("ui_intent", UIIntentCode.INTENT_WEIGHT_FOCUS) if isinstance(ui_val, str): ui_code = UIIntentCode.INTENT_WEIGHT_FOCUS if "Focus" in ui_val else 0 else: ui_code = int(ui_val) media_hash = bytes.fromhex(data.get("media_sha256_hex", "00" * 32)) return struct.pack(cls.CONTRACT_FMT, int(data.get("ttl_sec", 3600)), int(data.get("zone", MemoryZone.ZONE_1_ACTIVE_STREAM)), int(data.get("hvac_mode", HVACMode.SOLAR_SHIELD_SUMMER)), ui_code, int(data.get("laser_pulse_cap_mw", 850)), int(data.get("reserved_flags", 0)), media_hash,)

```
elif node_type == UniNodeType.GENESIS_TEXT: return data if isinstance(data, bytes) else
str(data).encode("utf-8")
elif node_type == UniNodeType.HOLO_WAVEFRONT: buf = bytearray(len(data) * 20) offset = 0
for v in data: struct.pack_into( cls.HOLO_FMT, buf, offset, float(v["x"]), float(v["y"]), float(v["z"]),
float(v["phase_2pi"]), int(v["rgb565"]), int(v["pulse_mw"]), ) offset += 20 return bytes(buf)
raise ValueError(f"Unsupported UniNodeType: {node_type}")
@classmethod def unpack_node(cls, node_type: UniNodeType, raw_buf: Union[bytes,
memoryview]) -> Any: view = memoryview(raw_buf)
if node_type == UniNodeType.NOTARY_ANCHOR: root, pub, sig, ts_us, delta_t, st =
struct.unpack(cls.NOTARY_FMT, view) return { "data_merkle_root_256": root.hex(),
"signer_pubkey_ed25519": pub.hex(), "signature_ed25519": sig.hex(), "timestamp_us": ts_us,
"cooling_off_delta_t_sec": delta_t, "status": "PENDING_DELTA_T_CONFIRMATION" if st == 1
else "FINALIZED", }
elif node_type == UniNodeType.SEMANTIC_TENSOR: # Zero-Copy cast to IEEE 754 float32
view return list(view.cast("f"))
elif node_type == UniNodeType.KNOWLEDGE_GRAPH: return
list(cls.iter_triplets_zero_copy(view))
elif node_type == UniNodeType.TELEMETRY_MATRIX: out: Dict[str, float] = {} for m_id, val in
struct.iter_unpack(cls.TELEMETRY_FMT, view): out[resolve_metric_id32(m_id)] = round(val, 4)
return out
elif node_type == UniNodeType.BEHAVIOR_CONTRACT: ttl, zone, hvac, ui_code, mw, flags,
m_hash = struct.unpack(cls.CONTRACT_FMT, view) return { "ttl_sec": ttl, "zone": zone,
"hvac_mode": HVACMode(hvac).name, "ui_intent_code": f"0x{ui_code:04x}",
"laser_pulse_cap_mw": mw, "reserved_flags": flags, "media_sha256_hex": m_hash.hex(), }
elif node_type == UniNodeType.GENESIS_TEXT: return bytes(view).decode("utf-8")
elif node_type == UniNodeType.HOLO_WAVEFRONT: voxels = [] for x, y, z, ph, rgb, mw in
struct.iter_unpack(cls.HOLO_FMT, view): voxels.append({ "x": round(x, 4), "y": round(y, 4), "z":
round(z, 4), "phase_2pi": round(ph, 6), "rgb565": rgb, "pulse_mw": mw, }) return voxels
raise ValueError(f"Unknown UniNodeType during unpacking: {node_type}")
@classmethod def iter_triplets_zero_copy(cls, raw_view: memoryview) -> Iterator[Dict[str, Any]]:
"""Iterates over 32-byte binary triplets directly in memoryview without intermediate lists."""" for
s_id, p_id, o_id, weight, ego, flags in struct.iter_unpack(cls.TRIPLET_FMT, raw_view): yield {
"s": resolve_semantic_id64(s_id), "p": resolve_semantic_id64(p_id), "o":
resolve_semantic_id64(o_id), "s_id64": s_id, "p_id64": p_id, "o_id64": o_id, "weight":
round(weight, 4), "ego_alignment": round(ego, 4), "is_delta_conflict": bool(flags & 1), }
```

==========

==========

===

5. UNIVERSAL .UNI CONTAINER ENGINE

(O(1) GATE, MERKLE, ZERO-COPY)

==========

==========

===

class UniUniversalContainer: """ 32-Byte Little-Endian UniHeader Layout ('<4sHHIIQ4s4s'): 00..03 (4B) MAGIC_BYTES : b'UNI8' 04..05 (2B) PROTO_FLAGS : Version (0x0820) & mode bitmask 06..07 (2B) NODE_BITMAP : Active node bitmask (nodes 0..15) 08..11 (4B) PAYLOAD_LIMIT : Exact encrypted node stream size in bytes 12..15 (4B) SESSION_SALT : 32-bit TRNG cryptographic salt 16..23 (8B) TIMESTAMP_US : 64-bit UTC microsecond counter 24..27 (4B) MERKLE_TAG : Truncated 4-byte Data Merkle root anchor 28..31 (4B) HEADER_ECC : 4-byte Blake2s-32 header integrity syndrome """ HEADER_FMT = "<4sHHIIQ4s4s" HEADER_SIZE = 32 MAGIC = b"UNI8" NODE_OVERHEAD = 35 # 7B descriptor + 12B IV + 16B GCM Tag
@staticmethod def leaf_hash(node_id: int, node_type: int, raw_bytes: Union[bytes, memoryview]) -> bytes: h = hashlib.sha256(b"UNI8_LEAF:" + struct.pack("<HB", -
@staticmethod branch_hash(left: bytes) bytes, def h.digest() h.update(raw_bytes) node_id, node_type)) return right:> bytes: return hashlib.sha256(b"UNI8_BRANCH:" + left + right).digest()
@classmethod def build_merkle_tree(cls, node_ids: List[int], leaves: List[bytes]) -> Tuple[bytes, Dict[int, List[Dict[str, str]]]]: """Builds a SHA-256 Merkle Tree, returning the 32-byte root and per-node Branch Proofs.""" if not leaves: return b"\x00" * 32, {} proofs: Dict[int, List[Dict[str, str]]] = {nid: [] for nid in node_ids} positions = {nid: idx for idx, nid in enumerate(node_ids)} level = list(leaves)
while len(level) > 1: if len(level) % 2 == 1: level.append(level[-1]) for nid, pos in positions.items(): sib = level[pos + 1] if pos % 2 == 0 else level[pos - 1] side = "right" if pos % 2 == 0 else "left" proofs[nid].append({"side": side, "hash": sib.hex()}) positions[nid] = pos // 2 level = [cls.branch_hash(level[i], level[i + 1]) for i in range(0, len(level), 2)]
return level[0], proofs
@classmethod def verify_header_o1(cls, buffer: Union[bytes, memoryview]) -> bool: """Constant-time O(1) validation of the 32-byte header and payload size limit.""" view = memoryview(buffer) if len(view) < cls.HEADER_SIZE: return False magic, flags, bitmap, payload_limit, salt, ts_us, m_tag, ecc = struct.unpack(cls.HEADER_FMT, view[:cls.HEADER_SIZE]) if magic != cls.MAGIC: return False actual_rem = len(view) - cls.HEADER_SIZE if (flags & ProtoFlags.RS_FEC_APPENDED) and actual_rem >= payload_limit: pass elif actual_rem != payload_limit: return False expected_ecc = hashlib.blake2s(view[:28], digest_size=4).digest() return hmac.compare_digest(ecc, expected_ecc)
@classmethod def create_container(cls, nodes_input: Dict[int, Tuple[UniNodeType, Any]], keyring: SovereignKeyring, cooling_off_seconds: int = 0, under_duress: bool = False, is_delta_frame: bool = False, spatial_broadcast: bool = False, rs_fec_tail: bytes = b"",) -> bytes: """Assembles an encrypted pure-binary .uni container with 39-byte Header+NodeDesc AAD.""" if

0 in nodes_input: raise ValueError("Node ID 0 is reserved for the system NOTARY_ANCHOR node.") for nid in nodes_input: if not (1 <= nid <= 15): raise ValueError(f"Node ID {nid} exceeds 16-bit NODE_BITMAP bounds (1..15).")
session_salt = struct.unpack("<I", "<4sHHIIQ4s", "cooling_off_delta_t_sec": "data_merkle_root_256": "signature_ed25519": "signer_pubkey_ed25519": "timestamp_us":) * + - 1_000_000) << <HBI," <I"," @classmethod Dict[int, List[bytes]="[]" ProtoFlags.TAMPER_MESH_INTACT SovereignKeyring) Tuple[UniNodeType, Union[bytes, *="cls.build_merkle_tree(sorted_user_ids," aad="full_header_32b" all_ids="[0]" all_ids) all_ids: b"".join(encrypted_chunks) bitmap="sum(1" bitmap, bytes]]="{}" cls.MAGIC, content="nodes_input[nid]" content) cooling_off_seconds) cooling_off_seconds, ct) ct, data_merkle_root, data_merkle_root.hex(), data_merkle_root[:4], def digest_size="4).digest()" ecc else: encrypted_chunks.append(node_desc encrypted_chunks: f"NODE*{nid}".encode()) flags for full_header_32b header_28b="struct.pack(" if in inspect_and_route(cls, int(flags), int(ntype), is_delta_frame: iv iv, keyring: len(packed_nodes[nid][1]) len(raw_b)) memoryview], nid nid) nid, node_desc="struct.pack(" node_desc) node_key="hkdf_sha256(keyring.master_encryption_secret," notary_dict="{" notary_dict) notary_msg="data_merkle_root" notary_raw_b="UniNodePayload.pack_node(UniNodeType.NOTARY_ANCHOR," notary_raw_b) ntype, os.urandom(4))[0] os.urandom(8)).digest()[:12] packed_nodes: packed_nodes[0]="(UniNodeType.NOTARY_ANCHOR," packed_nodes[nid]="(ntype," payload_limit="sum(cls.NODE_OVERHEAD" payload_limit, raw_b="packed_nodes[nid]" raw_b) raw_b)) raw_b, return rs_fec_tail rs_fec_tail: salt_bytes="struct.pack(" salt_bytes, session_salt) session_salt, signature="keyring.sign_data(notary_msg," signature.hex(), signer_pubkey, signer_pubkey.hex(), sorted_user_ids sorted_user_ids: spatial_broadcast: struct.pack("<H", struct.pack("<QI", tag timestamp_us="int(time.time()" timestamp_us, under_duress="under_duress)" uni_binary: user_leaves) user_leaves.append(cls.leaf_hash(nid, user_leaves: |="ProtoFlags.STREAM_DELTA_FRAME" }> Dict[str, Any]: """Zero-Copy container inspection via memoryview slicing, Notary verification & Zone routing.""" view = memoryview(uni_binary) if not cls.verify_header_o1(view): raise ValueError("O(1) Hardware Gate: Invalid header or payload length mismatch!")
full_header_32b = bytes(view[:cls.HEADER_SIZE]) magic, flags, bitmap, payload_limit, session_salt, ts_us, m_tag, ecc = struct.unpack(cls.HEADER_FMT, full_header_32b)
salt_bytes = struct.pack("<I", "FAST_DELTA_FRAME_120HZ", "KEYFRAME_NOTARIZED", "VERIFIED_SESSION_AEAD: "assigned_memory_zone": "data": "frame_mode": "internal_security_verdict": "nodes": "public_signature_valid": "raw_size_bytes": "type": &) + - 19:offset 19] 35 35:offset 35] 7:offset 7]) < @classmethod Any]]="{}" Delta Dict[int, Dict[str, Frame.", GCM Header-bound MemoryZone.ZONE_1_ACTIVE_STREAM.name, ProtoFlags.STREAM_DELTA_FRAME: SovereignKeyring, True, UniNodePayload.unpack_node(ntype, Union[bytes, aad="full_header_32b" cls, cooling_off_delta_t_sec"]" ct_view="view[offset" ct_view, data"]" data_merkle_root_256"])" data_root="bytes.fromhex(notary[" def delta_t="notary[" delta_t) elif else: export_selective_slice(f"NODE_{nid}".encode()) flags hmac.compare_digest(signer_pub, if int is_sig_valid="SovereignKeyring.verify_signature(signer_pub," is_sig_valid, is_sig_valid: iv_view="view[offset" iv_view, keyring.duress_pubkey): keyring: len(raw_b), memoryview(raw_b)), memoryview], nid, node_desc="bytes(view[offset:offset" node_desc) node_key="hkdf_sha256(keyring.master_encryption_secret," not notary="unpacked_nodes[0][" notary["timestamp_us"], ntype="UniNodeType(ntype_int)" ntype.name, ntype_int, offset payload_limit raw_b="aead_decrypt(node_key," raw_len raw_len] return salt_bytes,

session_salt) signature="bytes.fromhex(notary[" signature) signature_ed25519"])" signed_msg="data_root" signed_msg, signer_pub="bytes.fromhex(notary[" signer_pubkey_ed25519"])" stream_end="cls.HEADER_SIZE" stream_end: struct.pack("<QI", tag_view="view[offset" tag_view, target_node_id: uni_binary: unpacked_nodes, unpacked_nodes: unpacked_nodes[nid]="{" verdict="VERIFIED_SOVEREIGN: Authentic Primary Signature." verdict, while zone="MemoryZone.ZONE_1_ACTIVE_STREAM" zone.name, { }> Dict[str, Any]: """Exports a single decrypted user node accompanied by its Merkle Branch Proof & Node 0 Notary Seal.""" if target_node_id == 0: raise ValueError("Node 0 is the Notary Anchor; specify a user node (1..15) to disclose.") view = memoryview(uni_binary) if not cls.verify_header_o1(view): raise ValueError("Container failed O(1) header check.") full_header_32b = bytes(view[:cls.HEADER_SIZE]) _, flags, bitmap, payload_limit, session_salt, _, _, _ = struct.unpack(cls.HEADER_FMT, full_header_32b) if not (bitmap & (1 << target_node_id)): raise ValueError(f"Node {target_node_id} is not present in container.") salt_bytes = struct.pack("<I", "data_merkle_root_256": "decoded_data": "merkle_branch_proof": "node_id": "node_type": "node_type_name": "notary_anchor": "raw_payload_hex":) + - 0: 19:offset 19] 35 35:offset 35] 7:offset 7]) < @classmethod Any]="{}" Any], Dict[str, Optional[str]="None" UniNodePayload.unpack_node(target_type, UniNodeType.GENESIS_TEXT [] aad="full_header_32b" cls, ct_view="view[offset" ct_view, data_root_256, data_root_256.hex(), def else: expected_pubkey_hex: exported_slice: f"NODE_{nid}".encode()) if int(target_type), iv_view="view[offset" iv_view, memoryview(raw_b)) memoryview(target_raw)), nid="=" nid, node_desc="bytes(view[offset:offset" node_desc) node_key="hkdf_sha256(keyring.master_encryption_secret," notary_data="UniNodePayload.unpack_node(UniNodeType.NOTARY_ANCHOR," notary_data, notary_data: ntype_int, offset payload_limit proofs="cls.build_merkle_tree(user_ids," proofs[target_node_id], raw_b="aead_decrypt(node_key," raw_b)) raw_len raw_len] return salt_bytes, session_salt) stream_end="cls.HEADER_SIZE" stream_end: tag_view="view[offset" tag_view, target_node_id, target_node_id: target_raw="raw_b" target_raw, target_raw.hex(), target_type="UniNodeType(ntype_int)" target_type.name, user_ids, user_ids.append(nid) user_leaves="[]," user_leaves) user_leaves.append(cls.leaf_hash(nid, verify_selective_slice(while { }> bool: """Verifies a single exported node against the 256-bit Merkle root AND the author's Ed25519 signature.""" curr = cls.leaf_hash(exported_slice["node_id"], exported_slice["node_type"], bytes.fromhex(exported_slice["raw_payload_hex"]),) for step in exported_slice["merkle_branch_proof"]: sib = bytes.fromhex(step["hash"]) curr = cls.branch_hash(curr, sib) if step["side"] == "right" else cls.branch_hash(sib, curr) notary = exported_slice["notary_anchor"] if not hmac.compare_digest(curr.hex(), notary["data_merkle_root_256"]): return False
if expected_pubkey_hex and not hmac.compare_digest(notary["signer_pubkey_ed25519"], expected_pubkey_hex): return False
signed_msg = (bytes.fromhex(notary["data_merkle_root_256"]) + struct.pack("<QI", """ #="==="
(RAM) - 2 32-byte 6. @staticmethod BINARY Conflict Delta GRAPH Graph Knowledge MERGING) RAM. SEMANTIC SovereignKeyring.verify_signature(Superposition TRIAGE Union[List[float], ZERO-COPY ZONE Zero-Copy Zone2SemanticTriage: and background binary bytes.fromhex(notary["signature_ed25519"]), bytes.fromhex(notary["signer_pubkey_ed25519"]), class clustering cosine_distance(vec_a: def in memoryview]) memoryview], merging notary["cooling_off_delta_t_sec"]) notary["timestamp_us"], return signed_msg, tracking vec_b: vector with> float: a_floats = vec_a.cast("f") if isinstance(vec_a, memoryview) else vec_a b_floats = vec_b.cast("f") if isinstance(vec_b, memoryview) else vec_b dot = sum(x * y for x, y in

zip(a_floats, b_floats)) norm_a = math.sqrt(sum(x * x for x in a_floats)) norm_b = math.sqrt(sum(y * y for y in b_floats)) if norm_a == 0.0 or norm_b == 0.0: return 1.0 return round(1.0 - (dot / (norm_a * norm_b)), 6)
@classmethod def merge_binary_graphs_in_ram(cls, view_a: memoryview, view_b: memoryview) -> bytes: """ Merges two 32-byte aligned (<QQQeeI) """ (s_id, (view_a, 64-bit Dict[Tuple[int, Graph Knowledge List[Tuple[int, []) _ any(existing[0]="=" binary buffers comparison conflict_flag="1" directly edges ego)) ego, existing float, float]]]="{}" fmt="UniNodePayload.TRIPLET_FMT" for grouped.items(): grouped.setdefault(key, grouped: grouped[key]): grouped[key].append((o_id, if in int], integer key="(s_id," len(edges) not o_id o_id, out_buf="bytearray()" p_id) p_id), p_id, parsing! register s_id, string struct.iter_unpack(fmt, via view view): view_b): w, zero —> 1 else 0 for o_id, w, ego in edges: out_buf.extend(struct.pack(fmt, s_id, p_id, o_id, w, ego, conflict_flag)) return bytes(out_buf)
@classmethod def merge_knowledge_graphs(cls, graph_a: Union[List[Dict[str, Any]], bytes, memoryview], graph_b: Union[List[Dict[str, Any]], bytes, memoryview],) -> List[Dict[str, Any]]: """High-level wrapper that packs (if needed) and merges via the Zero-Copy binary RAM engine.""" raw_a = UniNodePayload.pack_node(UniNodeType.KNOWLEDGE_GRAPH, graph_a) if isinstance(graph_a, list) else bytes(graph_a) raw_b = UniNodePayload.pack_node(UniNodeType.KNOWLEDGE_GRAPH, graph_b) if isinstance(graph_b, list) else bytes(graph_b) merged_bytes = cls.merge_binary_graphs_in_ram(memoryview(raw_a), memoryview(raw_b)) return list(UniNodePayload.iter_triplets_zero_copy(memoryview(merged_bytes)))

==========

==========

===

SELF-TEST & VERIFICATION SUITE (ALL 5 CORE, ZERO-COPY & HOLO TESTS)

==========

==========

===

if **name** == "**main**": print("=") print(" .uni Universal Reference SDK v8.2 - Pure Binary Zero-Copy Suite") print(f" Crypto Backend : {'Hardware AES-256-GCM & Ed25519' if HAS_CRYPTO_LIB else 'Stdlib AEAD Fallback'}") print(" JSON Dependency: ELIMINATED (0% Text Parsing inside Operational Nodes)") print("=\n")
keyring = SovereignKeyring(master_seed=b"SOVEREIGN_ROOT_SEED_2026_V8_2")
sample_nodes = { 1: (UniNodeType.SEMANTIC_TENSOR, [0.82, 0.14, -0.33, 0.91]), 2:

(UniNodeType.KNOWLEDGE_GRAPH, [{"s": "MZM-Pod", "p": "uses_memory", "o": "Volatile_SRAM", "weight": 0.99, "ego_alignment": 0.95}, {"s": "Dump_Valve", "p": "triggers_at", "o": "1.2_atm", "weight": 0.95, "ego_alignment": 0.90},]), 3: (UniNodeType.TELEMETRY_MATRIX, {"carotid_IMT_mm": 0.72, "VAT_SAT_ratio": 0.41, "PWV_ms": 6.8}), 4: (UniNodeType.BEHAVIOR_CONTRACT, { "ttl_sec": 3600, "zone": MemoryZone.ZONE_1_ACTIVE_STREAM, "hvac_mode": HVACMode.SOLAR_SHIELD_SUMMER, "ui_intent": UIIntentCode.INTENT_WEIGHT_FOCUS, "laser_pulse_cap_mw": 850, }), 5: (UniNodeType.GENESIS_TEXT, "# Session Summary\nVascular and metabolic parameters within normal range."), 7: (UniNodeType.HOLO_WAVEFRONT, [{"x": 12.5, "y": 40.0, "z": 2.5, "phase_2pi": 3.14159, "rgb565": 0xF800, "pulse_mw": 850}, {"x": 12.6, "y": 40.0, "z": 2.8, "phase_2pi": 1.57079, "rgb565": 0x07E0, "pulse_mw": 820},]), }
Test 1: Pure Binary Container with 12-hour (43,200s) Delta-T Cooling-Off Timelock uni_normal = UniUniversalContainer.create_container(sample_nodes, keyring, cooling_off_seconds=43200, under_duress=False) inspected_normal = UniUniversalContainer.inspect_and_route(memoryview(uni_normal), keyring) print(f"[Test 1] Pure Binary Container Packed ({len(uni_normal)} bytes total)") print(f" Node 0 Binary Size : {inspected_normal['nodes'][0]['raw_size_bytes']}B (Fixed <32s32s64sQIB)") print(f" Node 2 Binary KG Size : {inspected_normal['nodes'][2]['raw_size_bytes']}B (2 triplets x 32B <QQQeeI)") "" "1.5_atm", "Dump_Valve", "ego_alignment": "o": "p": "triggers_at", "weight": # & (< (Fixed (Node (Nodes (Signed (Steganographic))" + - -0.31, -0.33, 0.14, 0.15 0.15, 0.80, 0.85}], 0.89]) 0.91], 1, 2 2, 2: 2] 3 3: 3] 4 4, 48B 4: 4] 5, 7 : <IBBHII32s)") <If)") Action Assigned Binary Coerced Conflict Container Contract Cosine Delta Disclosed Disclosure Distance="{c_dist}" Duress Ed25519 Export External Key) Key)") Memory Merkle Metrics Node Notary Proof Public RAM Sees Selective Semantic Signature Single-Node Sovereign Status Steganographic Superposition TELEMETRY_MATRIX Telemetry) Test Triage Valid="{inspected_duress['public_signature_valid']}" Verifier Zero-Copy Zone [0.80, [{"s": [{triplet['s']}] c_dist="Zone2SemanticTriage.cosine_distance([0.82," cooling_off_seconds="0," delta_tag=" [SUPERPOSITION CONFLICT]" else expected_pubkey_hex="keyring.primary_pubkey.hex()" for if in inspected_duress="UniUniversalContainer.inspect_and_route(memoryview(uni_duress)," keyring) keyring, merged="Zone2SemanticTriage.merge_knowledge_graphs(" merged: print("[Test print(f" print(f"[Test remain sample_nodes, sample_nodes[2][1], slice_verified="UniUniversalContainer.verify_selective_slice(" target_node_id="3)" telemetry_slice="UniUniversalContainer.export_selective_slice(memoryview(uni_normal)," telemetry_slice, threshold)") triplet triplet["is_delta_conflict"] under_duress="True" unexposed)\n") uni_duress="UniUniversalContainer.create_container(" with {inspected_duress['internal_security_verdict']}\n") {inspected_normal['assigned_memory_zone']}") {inspected_normal['nodes'][0]['data']['status']}\n") {inspected_normal['nodes'][4]['raw_size_bytes']}B {inspected_normal['public_signature_valid']}") {slice_verified} {telemetry_slice['decoded_data']}") |> [{triplet['p']}] -> [{triplet['o']}] (w={triplet['weight']}){delta_tag}") print()
Test 5: Holo-Frame 120Hz Delta-Stream & Reed-Solomon Facade Broadcast delta_pkt = UniUniversalContainer.create_container({7: sample_nodes[7]}, keyring, is_delta_frame=True, spatial_broadcast=True, rs_fec_tail=b"\xAA" * 32,) res_df = UniUniversalContainer.inspect_and_route(memoryview(delta_pkt), keyring) print(f"[Test 5] Holo-Frame 120Hz Delta Packet ({len(delta_pkt)} bytes inc. 32B RS-FEC tail)") print(f" Frame

Mode : {res_df['frame_mode']}") print(f" Decoded Wavefront : {len(res_df['nodes'][7]['data'])} Pockels/MEMS voxels (20B/voxel)")
